

PROJECT: Real-Time Data Processing with Kinesis and Redshift

- **Services:** Kinesis, Redshift, Lambda, S3
- **Description:** Ingest streaming data from a simulated IoT device into Kinesis. Process the data using Lambda, store it in S3 for backup, and load it into Redshift for analysis.
- **Skills:** Data streaming, ETL, data warehousing.

Definition: This project is a real-time data processing pipeline built using AWS services where simulated IoT device data is streamed into Amazon Kinesis, processed using AWS Lambda, and stored in Amazon S3 for backup. The processed data is then loaded into Amazon Redshift for analysis and reporting. It demonstrates end-to-end data streaming, ETL processing, and data warehousing for real-time analytics use cases.

Scope of the Project: The scope of this project covers building an end-to-end real-time data processing system using AWS cloud services. It includes streaming IoT data ingestion through Amazon Kinesis, real-time processing using AWS Lambda, reliable storage in Amazon S3, and analytical querying in Amazon Redshift. This system can be extended for use in industries like IoT monitoring, smart cities, and predictive analytics, where continuous data flow and fast decision-making are required.

Methodologies:

- **Data Generation:** Simulated IoT device generates real-time streaming data (temperature, humidity, timestamp).
- **Data Ingestion:** Amazon Kinesis collects and streams incoming IoT data in real time.
- **Data Processing:** AWS Lambda processes the streaming data, applies filtering or transformations.
- **Data Storage (Backup):** Processed data is stored in Amazon S3 for backup and durability.
- **Data Loading (ETL):** Data from S3 is loaded into Amazon Redshift using the COPY command.
- **Data Warehousing:** Amazon Redshift stores structured data for fast querying and analytics.
- **Data Analysis:** SQL queries are used to analyze trends, detect anomalies, and generate insights.

Services Used in this Project:

Amazon Kinesis is used for real-time data streaming and ingestion.

AWS Lambda is used for processing and transforming streaming data.

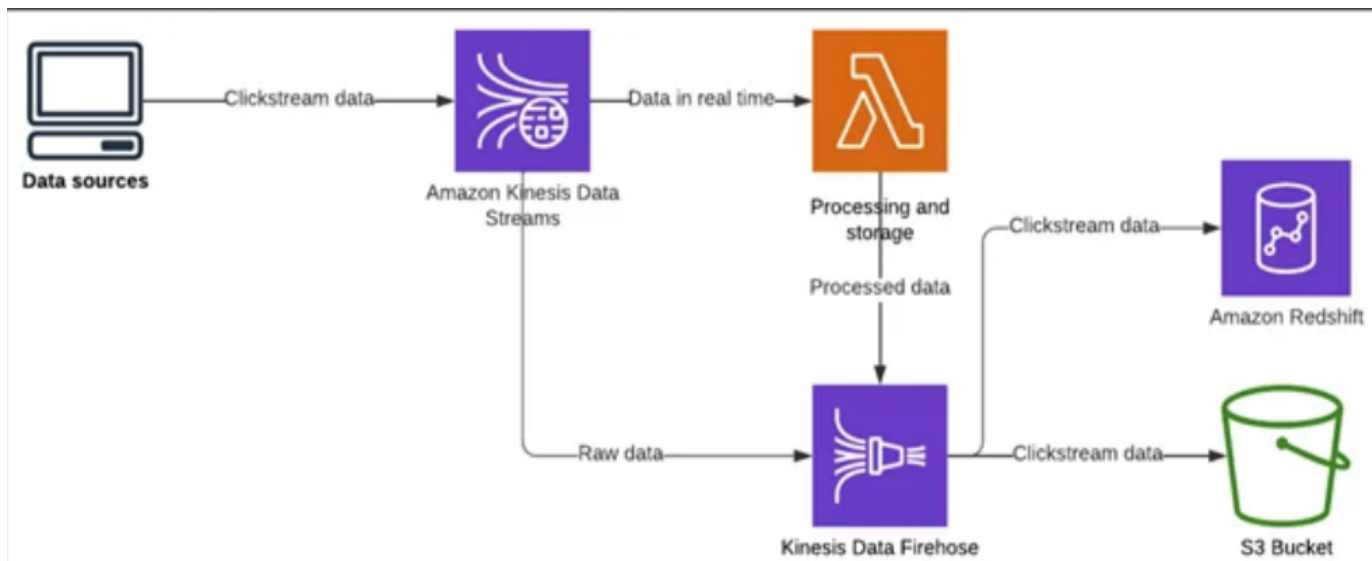
Amazon S3 is used for storing processed data as backup.

Amazon Redshift is used as a data warehouse for analytics and querying.

AWS IAM is used for managing secure access and permissions between services.

Amazon CloudWatch is used for monitoring logs and system performance.

Architecture Diagram:



Implementation and Execution Real-Time Data Processing with Kinesis and Redshift:

STEP-1: Created Kinesis

Go to **Kinesis** → **Data Streams**

Click **Create stream**

Name: **iot-stream**

Shards: **1**

Amazon Kinesis > Data streams

Data streams (1/1) Info Process data in real time Create a Firehose stream Actions Create data stream

☐ Find data streams

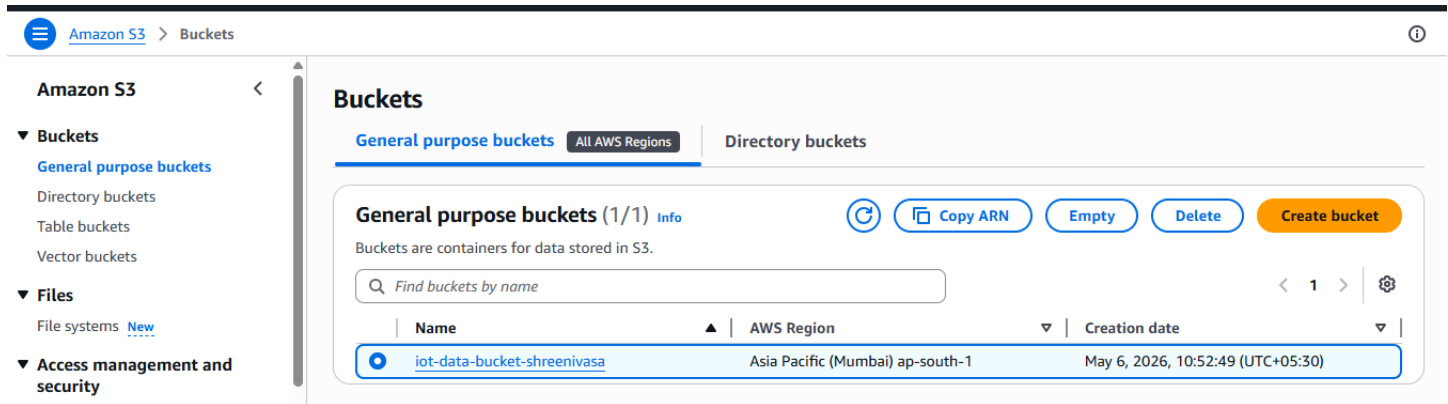
☒	Name	Status	Capacity mode	Maximum record size	Warm throughput
☒	iot-stream	Active	On-demand	1024 KiB	Not set

```
[root@ip-172-31-1-204 ec2-user]# python3 producer.py
/usr/local/lib/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April 29, 2026
. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found here:
https://aws.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
warnings.warn(warning, PythonDeprecationWarning)
Response: {'ShardId': 'shardId-000000000000', 'SequenceNumber': '4967428598061136812004846566627666524200129016337793058', 'ResponseMetadata': {'Re
questId': 'f04c69f8-4aa5-80a9-9238-9f2fbf05de5a', 'HTTPStatusCode': 200, 'HTTPHeaders': {'x-amzn-requestid': 'f04c69f8-4aa5-80a9-9238-9f2fbf05de5a',
'x-amz-id-2': 'CET2S6i80izvJ3DCGirS5wJxOgVIRVr8ehSzelP9dnd35A0h0ANP15BLD5HaHmoc2ayE5IB/cAyriML6nN2I9Vkg/9VEgKOfGOzinRuKSA=', 'date': 'Wed, 06 May
2026 07:58:15 GMT', 'content-type': 'application/x-amz-json-1.1', 'content-length': '110', 'connection': 'keep-alive', 'RetryAttempts': 0}}
Sent: {'deviceId': 'sensor-3', 'temperature': 34.04, 'humidity': 63.26, 'timestamp': '2026-05-06T07:58:15.014130'}
Response: {'ShardId': 'shardId-000000000000', 'SequenceNumber': '49674286196148070463846578316597954147203771115256152114', 'ResponseMetadata': {'Re
questId': 'd81008a8-5c03-a0aa-ba64-fe71a9a3fc59', 'HTTPStatusCode': 200, 'HTTPHeaders': {'x-amzn-requestid': 'd81008a8-5c03-a0aa-ba64-fe71a9a3fc59',
'x-amz-id-2': 'weP3vOtOXXKQgVizA91lxrwcGuzKnp7ha67k6Yngmq8GedzSdFKQzXs01nT7LwnB0zoTkDhLZblgg3opZ3WgBroxZWKQpvgdLoElNg06X+bs=', 'date': 'Wed, 06 May
2026 07:58:17 GMT', 'content-type': 'application/x-amz-json-1.1', 'content-length': '110', 'connection': 'keep-alive', 'RetryAttempts': 0}}
Sent: {'deviceId': 'sensor-1', 'temperature': 30.22, 'humidity': 40.11, 'timestamp': '2026-05-06T07:58:17.044137'}
Response: {'ShardId': 'shardId-000000000000', 'SequenceNumber': '49674285983064450091881826037810985543290280488248279042', 'ResponseMetadata': {'Re
questId': 'f8eaae25-d8ed-6dd0-9a9e-58fe2d4d3123', 'HTTPStatusCode': 200, 'HTTPHeaders': {'x-amzn-requestid': 'f8eaae25-d8ed-6dd0-9a9e-58fe2d4d3123',
'x-amz-id-2': 'MHsaBYzHXKiNep/xrfYHyxCGr5/vMFZTsvzeJktVewylWa81g2aK6DQ41+DB3Xgcq5JpHxmoVLW7Db6y49UnYm5jPuXe7ML0YzCtzAl38RU=', 'date': 'Wed, 06 May
2026 07:58:19 GMT', 'content-type': 'application/x-amz-json-1.1', 'content-length': '110', 'connection': 'keep-alive', 'RetryAttempts': 0}}
Sent: {'deviceId': 'sensor-5', 'temperature': 24.15, 'humidity': 67.69, 'timestamp': '2026-05-06T07:58:19.054926'}
```

STEP 2: Create S3 Bucket

Create bucket: `iot-data-bucket-shreenivasa`

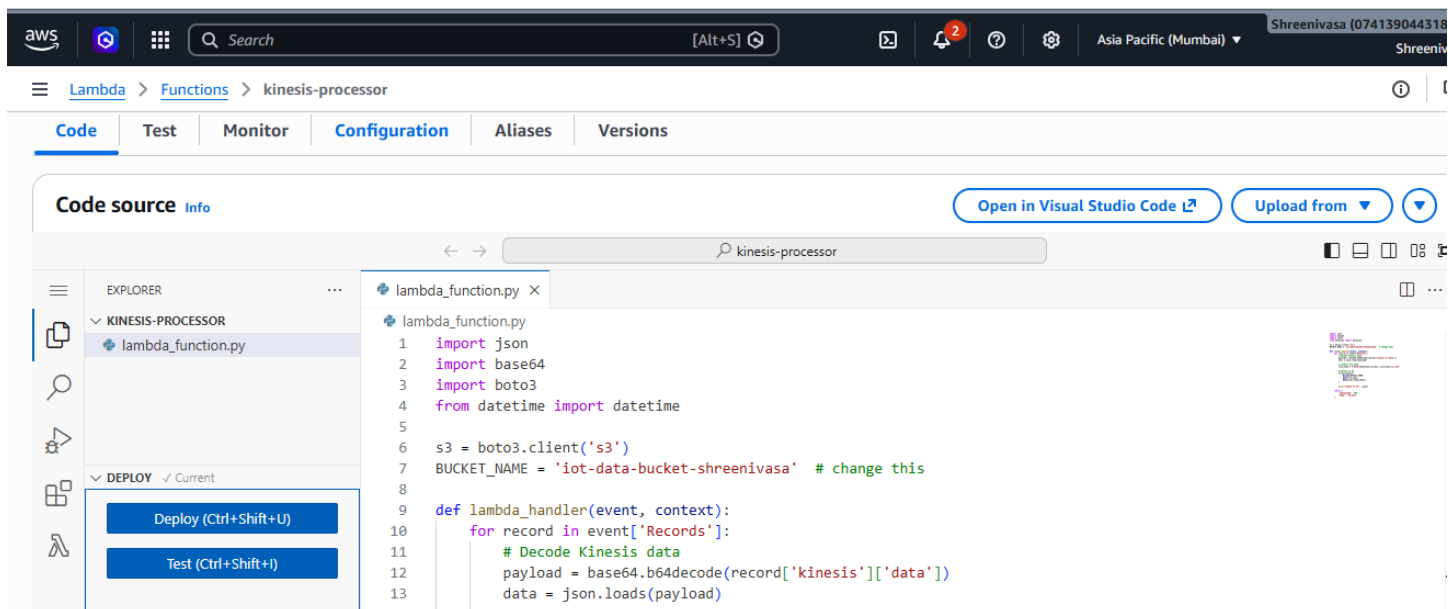
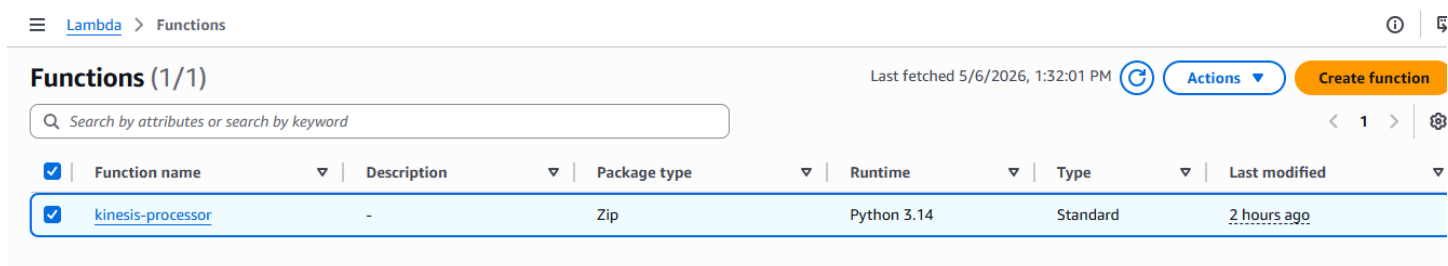
Enable default settings



STEP 3: Create Lambda Function

Name: `kinesis-processor`

Runtime: Python



STEP 5: Connect Kinesis → Lambda

- Open Lambda → Add Trigger
- Choose **Kinesis**
- Select: `iot-stream`
- Enable

AWS Lambda > Functions > kinesis-processor

kinesis-processor

Throttle Copy ARN Actions

Function overview info

Diagram Template

Function ARN: `arn:aws:lambda:ap-south-1:074139044318:function:kinesis-processor`

Description: -

Last modified: 3 hours ago

Export to Infrastructure Composer Download

Code Test Monitor **Configuration** Aliases Versions

Configuration

General configuration

Triggers

Permissions

Destinations

Function URL

Triggers (1) info

Search for triggers

Trigger

Kinesis: [iot-stream](#)
arn:aws:kinesis:ap-south-1:074139044318:stream/iot-stream
state: Enabled
Details

Check Lambda Logs

- Go to Lambda → Monitor → CloudWatch logs
- You should see:

CloudWatch > Log management > [/aws/lambda/kinesis-processor](#) > 2026/05/06/[\$LATEST]89c962957c37404ea6127a32912eac4b

CloudWatch

Favorites and recents

Ingestion

Dashboards

Alarms 0 0 0 0

AI Operations

GenAI Observability

Application Signals (APM)

Infrastructure Monitoring

Logs

Log Management

Log Analytics Preview

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

1m 1h Local timezone

Display

Timestamp	Message
	No older events at this moment. Retry
2026-05-06T13:28:15.231+05:30	INIT_START Runtime Version: python:3.14.v39 Runtime Version ARN: arn:aws:lambda:ap-south-1::runtime:06dae...
2026-05-06T13:28:15.740+05:30	START RequestId: 1d7a1a75-d25d-44d6-98e3-c266d6f6f67a Version: \$LATEST
2026-05-06T13:28:16.014+05:30	Saved to S3: {'deviceId': 'sensor-3', 'temperature': 34.04, 'humidity': 63.26, 'timestamp': '2026-05-06T0...
2026-05-06T13:28:16.034+05:30	END RequestId: 1d7a1a75-d25d-44d6-98e3-c266d6f6f67a
2026-05-06T13:28:16.034+05:30	REPORT RequestId: 1d7a1a75-d25d-44d6-98e3-c266d6f6f67a Duration: 293.87 ms Billed Duration: 803 ms Memory...
2026-05-06T13:28:17.083+05:30	START RequestId: 93bee14e-b079-4f64-bbec-f82fd3fa11c8 Version: \$LATEST

STEP 6: Create Redshift Cluster

AWS Redshift:

- Create cluster
- Name: `iot-redshift`
- Create database: `dev`
- Create table:

The screenshot shows the AWS Redshift console. On the left, the 'Amazon Redshift' sidebar is visible with the 'Clusters' tab selected. The main area displays the 'Clusters (1)' list. A table shows the cluster 'iot-redshift-cluster' with a status of 'Available' and a namespace of '322b87d4-9896-41e3...'. The 'Query data' tab is active, showing the cluster name in a dropdown and buttons for 'Copy JDBC URL' and 'Copy ODBC URL'. A 'Create cluster' button is visible in the top right of the cluster list area.

Cluster	Status	Cluster namespace	Average query dura...	Average num
iot-redshift-cluster ra3.4xlarge 2 nodes 244.1 TB	Available	322b87d4-9896-41e3...		

The screenshot shows the AWS Redshift query editor v2. The left sidebar shows the 'iot-redshift-cluster' selected. The main editor area contains a SQL query to create a table named 'iot_data'. The query is as follows:

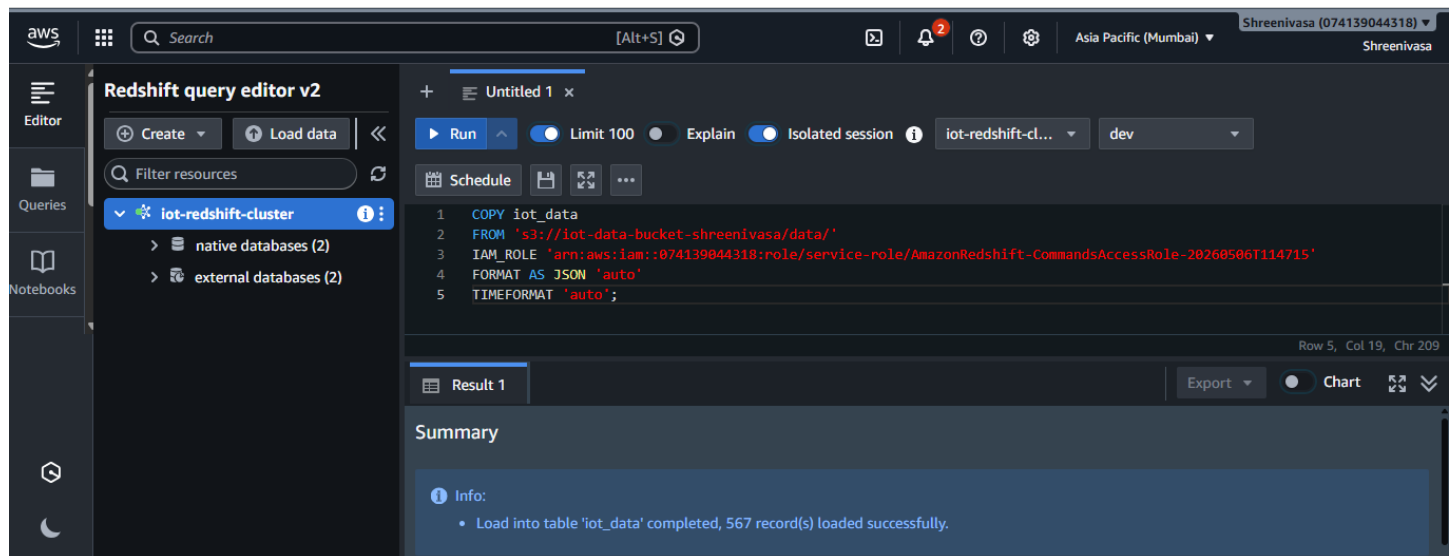
```
1 CREATE TABLE iot_data (  
2     device_id VARCHAR(50),  
3     temperature INT,  
4     humidity INT,  
5     timestamp TIMESTAMP DEFAULT GETDATE()  
6 );
```

The query is executed, and the 'Result 1' tab shows the execution details. The elapsed time is 0 seconds, and the result set query is the same as the one entered in the editor.

```
/* RQEV2-22h2VBVcrg */  
CREATE TABLE iot_data (  
    device_id VARCHAR(50),  
    temperature INT,  
    humidity INT,  
    timestamp TIMESTAMP DEFAULT GETDATE())
```

Table created,

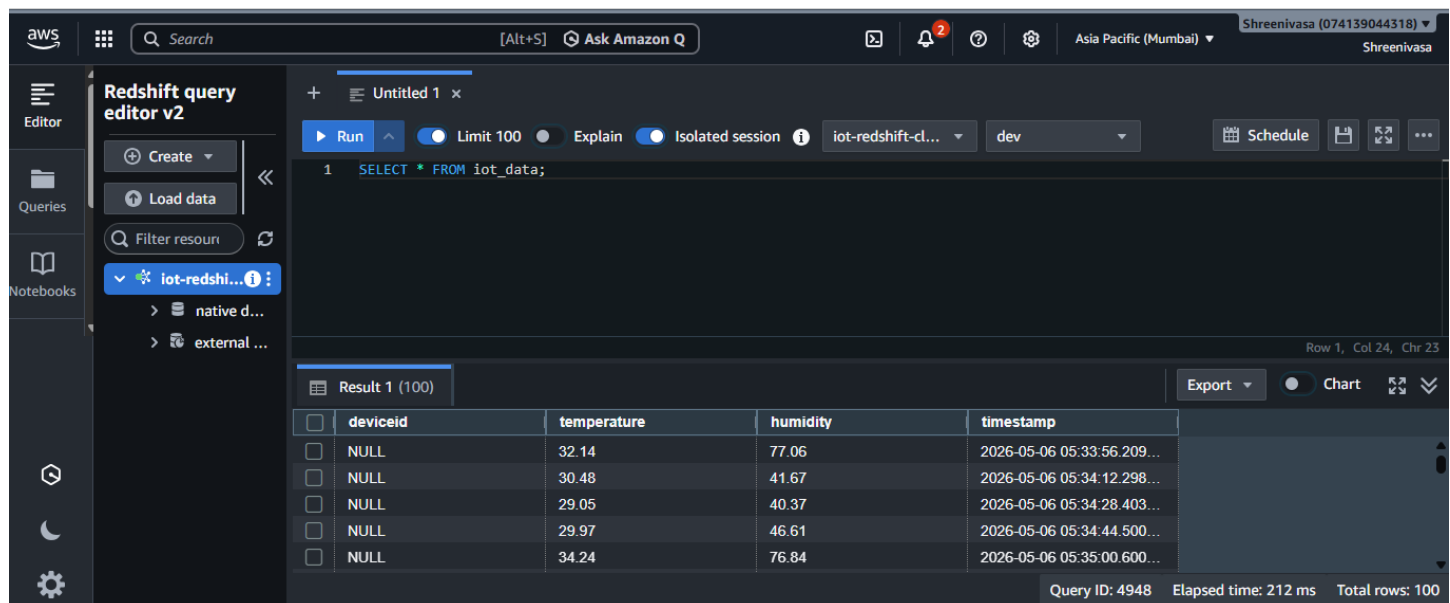
STEP 7: Load Data from S3 to Redshift



The screenshot shows the AWS Redshift query editor v2 interface. The left sidebar contains navigation options: Editor, Queries, and Notebooks. The 'Queries' section is active, showing a list of queries with 'iot-redshift-cluster' selected. The main editor area displays a SQL query: `COPY iot_data FROM 's3://iot-data-bucket-shreenivasa/data/' IAM_ROLE 'arn:aws:iam::074139044318:role/service-role/AmazonRedshift-CommandsAccessRole-20260506T114715' FORMAT AS JSON 'auto' TIMEFORMAT 'auto';`. The query is titled 'Untitled 1'. Below the query, the 'Result 1' tab is active, showing a 'Summary' section with an 'Info' message: 'Load into table 'iot_data' completed, 567 record(s) loaded successfully.'

STEP 8: Verify Data in Redshift

SELECT * FROM iot_data;



The screenshot shows the AWS Redshift query editor v2 interface. The left sidebar contains navigation options: Editor, Queries, and Notebooks. The 'Queries' section is active, showing a list of queries with 'iot-redshift-cluster' selected. The main editor area displays a SQL query: `SELECT * FROM iot_data;`. The query is titled 'Untitled 1'. Below the query, the 'Result 1 (100)' tab is active, showing a table of data. The table has four columns: deviceid, temperature, humidity, and timestamp. The data is as follows:

deviceid	temperature	humidity	timestamp
NULL	32.14	77.06	2026-05-06 05:33:56.209...
NULL	30.48	41.67	2026-05-06 05:34:12.298...
NULL	29.05	40.37	2026-05-06 05:34:28.403...
NULL	29.97	46.61	2026-05-06 05:34:44.500...
NULL	34.24	76.84	2026-05-06 05:35:00.600...

At the bottom of the interface, the status bar shows: Query ID: 4948, Elapsed time: 212 ms, Total rows: 100.

Troubleshoot:

Lambda not triggering

Problem:

- Data in Kinesis but Lambda not running

Fix:

- Check Trigger is added to Lambda
- Ensure Kinesis stream is selected properly
- Check Lambda execution role permissions:
 - AWSLambdaKinesisExecutionRole

Conclusion:

The project Real-Time Data Processing using Kinesis, Lambda, S3, and Redshift successfully demonstrates an end-to-end AWS streaming data pipeline. IoT-generated data is continuously ingested into Amazon Kinesis, where it is processed in real time using AWS Lambda. The processed data is then stored in Amazon S3 as a backup and later loaded into Amazon Redshift for structured querying and analysis.